Republic of Yemen
Ministry of Higher Education and Scientific Research
Taiz University
Al-Saeed Faculty of Engineering and IT
Software Engineering Department

الجمهورية اليمنية
وزارة التعليم العالي والبحث العلمي
جامعـــة تعـــز
كلية السعيد للهندسة وتقنية المعلومات
قسم هندسة البرمجيات

Design and Implementation of a Web-Based Task Management System Using Laravel

Submitted by:

Basma Mohmmed Haza'a Ali

Safa Abdulhakim Ahmed Ghaleb

Zainab Mokhtar Ahmed Ali

Supervised By:

Eng. Anas Alsabri

Yemen-Taiz
2026-2025

Abstract

The Task Manager System is a web-based application developed using the Laravel framework to improve task organization and enhance productivity. The system provides users with a centralized platform for managing daily tasks efficiently through a secure and interactive interface.

The application allows authenticated users to create, update, delete, and track their tasks based on priority, status, and deadlines. It also provides search and filtering functionalities to simplify task retrieval and improve usability.

The system follows the Model-View-Controller (MVC) architecture, which ensures modularity, maintainability, and scalability. Security mechanisms such as authentication, email verification, and ownership-based access control are integrated to protect user data and restrict unauthorized access.

This project aims to solve the common problem of poor task organization and missed deadlines by offering a structured digital solution that improves time management and workflow efficiency.

Table of contents

Table of Figures

1. Introduction

Task management is an essential activity in both personal and professional life. Individuals often deal with multiple tasks, deadlines, and priorities, which can become difficult to organize using traditional methods such as paper notes or unstructured reminders. Inefficient task management can lead to missed deadlines, reduced productivity, and poor time organization.

With the growth of digital solutions, task management systems have become important tools for improving workflow and helping users stay organized. These systems allow users to manage their responsibilities in a more structured and efficient way.

The Task Manager System was developed as a web-based application to address these challenges. It enables users to create, update, delete, and monitor tasks through an organized interface. The system also allows users to classify tasks by status and priority, making task tracking easier.

This project was implemented using the Laravel framework, which provides strong support for routing, database management, authentication, and middleware. The use of the MVC architectural pattern improves code organization and makes the application easier to maintain and expand in the future.

2. Problem Statement

Managing daily tasks efficiently is a common challenge faced by many individuals, especially students and professionals who deal with multiple responsibilities. Traditional task management methods, such as writing tasks on paper or relying on memory, are often unreliable and unorganized.

Many people face problems such as forgetting important deadlines, losing track of unfinished tasks, and failing to prioritize tasks based on urgency. This can result in poor time management, reduced productivity, and increased stress.

In addition, existing simple reminder tools may not provide enough flexibility for organizing tasks by status, priority, or deadlines. Users need a more structured system that allows them to manage tasks effectively while ensuring privacy and security.

Therefore, there is a need for a dedicated task management system that provides users with an organized, secure, and user-friendly platform for creating, tracking, updating, and managing their tasks efficiently.

3. Project Objectives

The main objective of the Task Manager System is to provide users with an efficient and organized platform for managing their daily tasks. The system is designed to improve productivity and help users track their responsibilities in a structured way.

The specific objectives of this project are:

- To develop a web-based system for creating and managing tasks.
- To provide users with the ability to add, update, and delete tasks easily.
- To allow users to organize tasks based on priority and status.
- To enable task searching and filtering for easier task retrieval.
- To improve time management by setting deadlines for tasks.
- To ensure task privacy and security through authentication and authorization mechanisms.
- To provide a user-friendly interface for better usability and interaction.
- To create a scalable system that can be improved with future enhancements.

4. Technologies Used

The Task Manager System was developed using modern web technologies to ensure efficiency, maintainability, and security. Each technology plays an important role in building the system and supporting its functionality.

The following technologies were used in the project:

Laravel Framework

Laravel is the main framework used for building the application. It provides several built-in components used in this project, including:

- **Routing** for handling application URLs and requests.
- **Authentication** for user registration, login, and access control.
- **Middleware** for protecting routes and restricting unauthorized access.
- **Eloquent ORM** for database interaction and model relationships.
- **Blade Template Engine** for building dynamic user interfaces.
- **Migration System** for database structure management.

These components simplify development and improve code organization through the MVC architecture.

PHP

PHP is the programming language used for backend development. Laravel is built on PHP, which makes it suitable for handling server-side logic.

MySQL

MySQL is used as the database management system for storing user information and task data. It provides reliable data storage and supports relational database design.

Tailwind CSS

Tailwind CSS was used for designing responsive and modern user interfaces. It provides utility-based classes that simplify styling and improve frontend development efficiency.

5. System Architecture

The Task Manager System follows the **Model-View-Controller (MVC)** architectural pattern, which is one of the most common and effective software design patterns in web development. This architecture separates the application into three main components: Model, View, and Controller.

The MVC architecture improves code organization, maintainability, and scalability by separating responsibilities between system components.

Model

The Model is responsible for managing data and interacting with the database. In this project, the **Task Model** handles task-related data such as title, description, status, priority, and due date. It also manages relationships with the User model.

View

The View is responsible for displaying data to users. In this project, Laravel Blade templates are used to create user interfaces such as task lists, task creation forms, and profile pages.

Examples:

- task/index.blade.php
- task/create.blade.php
- task/edit.blade.php
- task/show.blade.php

Controller

The Controller acts as the bridge between the Model and the View. It processes user requests, applies business logic, and returns responses.

Examples:

- TaskController
- DashboardController
- ProfileController

Database

The system uses MySQL as the backend database for storing user and task information. The database is connected to the application through Laravel Eloquent ORM.

The MVC architecture ensures that the system remains structured, modular, and easier to maintain and expand in the future.

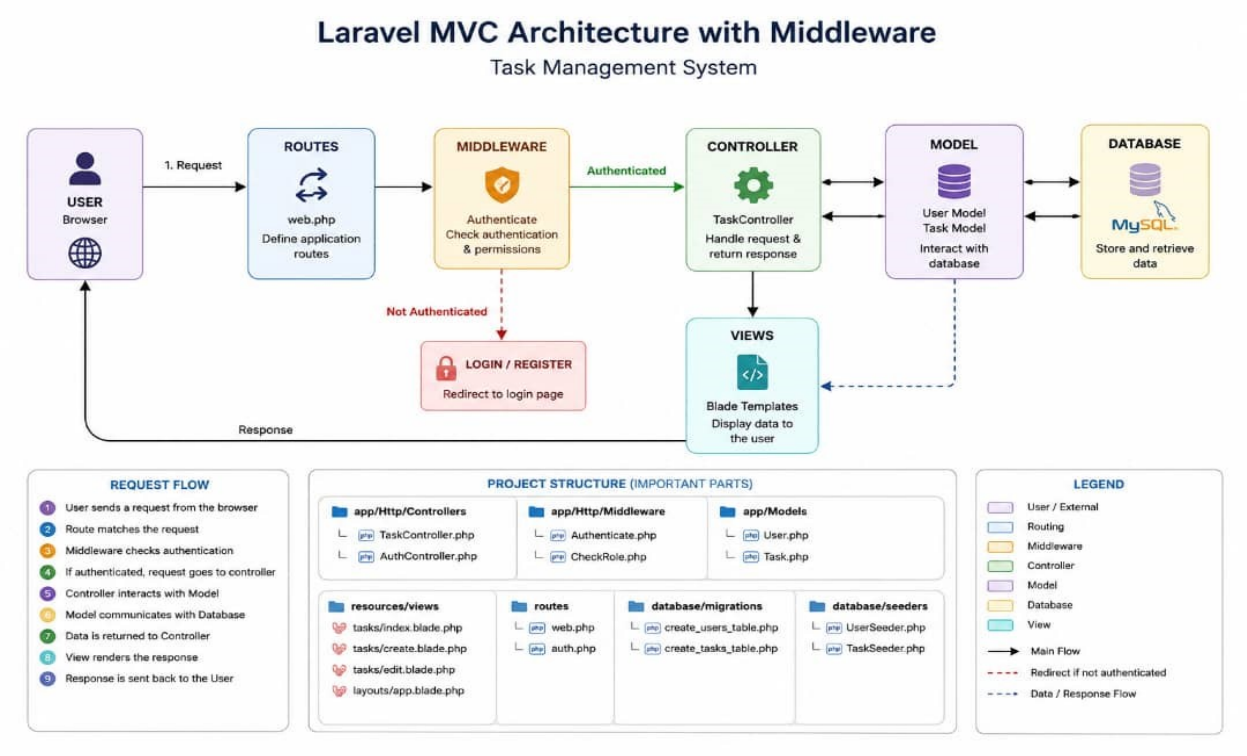

Figure 1: MVC Architecture of Task Management system

6. Security and Access Control

Security is one of the most important aspects of any web application, especially when handling personal user data. The Task Manager System applies multiple security mechanisms to ensure data privacy, protect user accounts, and prevent unauthorized access.

The system uses Laravel's built-in security features along with custom middleware to strengthen access control.

6.1 Authentication

Authentication ensures that only registered users can access the system. The project uses **Laravel Breeze** to implement the authentication system.

The authentication features include:

- User Registration
- User Login
- Password Reset
- Email Verification
- Logout

This ensures that only valid users can create and manage tasks.

6.2 Authorization

Authorization is used to define what actions users can perform after logging in. In this system, Laravel middleware is used to restrict access to authenticated and verified users only.

The following middleware are applied:

- **auth** → ensures the user is logged in.
- **verified** → ensures the email address is verified.

This improves the security level of the system and prevents unauthorized usage.

6.3 Ownership Protection

To protect user tasks, the system includes a custom middleware called **EnsureTaskOwner**.

This middleware checks whether the task belongs to the currently logged-in user before allowing access to:

- View task details
- Edit tasks
- Update tasks
- Delete tasks

This prevents users from accessing or modifying tasks that do not belong to them.

6.4 Data Validation

The system validates user input before storing or updating task data. Validation rules are applied inside the controller to ensure data consistency and prevent invalid entries.

Examples of validation include:

- Required task title
- Valid task status
- Valid priority level
- Due date validation

This helps reduce errors and maintain data integrity.

6.5 CSRF Protection

Laravel provides Cross-Site Request Forgery (CSRF) protection by default. All forms in the system are protected using CSRF tokens, preventing unauthorized form submissions.

This adds another layer of protection against malicious attacks.

7. Database Design and Relationships

The database is one of the core components of the Task Manager System because it stores all user information and task records. The system uses **MySQL** as the database management system due to its reliability, performance, and support for relational databases.

The database structure was designed to ensure data consistency, maintain relationships between entities, and support efficient task management operations.

7.1 Users Table

The **Users** table stores the information of registered users. Each user has a unique account used to access the system.

The main attributes of the Users table include:

- **id** → Unique identifier for each user.
- **name** → Stores the user's full name.
- **email** → Stores the email address.
- **password** → Stores the encrypted password.
- **created_at** → Stores account creation date.
- **updated_at** → Stores last update date.

This table is automatically managed through Laravel's authentication system.

7.2 Tasks Table

The **Tasks** table stores all tasks created by users. Each task belongs to one specific user.

The main attributes of the Tasks table include:

- **id** → Unique task identifier.
- **user_id** → Foreign key linked to the Users table.
- **title** → Stores the task title.
- **description** → Stores detailed task information.
- **status** → Defines the current task state.
- **priority** → Defines task importance level.
- **due_date** → Stores task deadline.
- **created_at** → Stores task creation date.
- **updated_at** → Stores last task update date.

This table is the main functional table of the system.

7.3 Relationship Between Tables

The relationship between Users and Tasks follows a **One-to-Many (1:N)** relationship:

- One user can create multiple tasks.
- Each task belongs to only one user.

This relationship is implemented in Laravel using Eloquent ORM:

- **User hasMany Tasks**
- **Task belongsTo User**

This design ensures proper data organization and ownership tracking.

7.4 Database Integrity

The system maintains database integrity by:

- Using primary keys for unique identification.
- Using foreign keys for relationship enforcement.
- Applying validation rules before data insertion.
- Restricting task access through ownership checks.

This design ensures that task records remain consistent, secure, and properly linked to their owners.

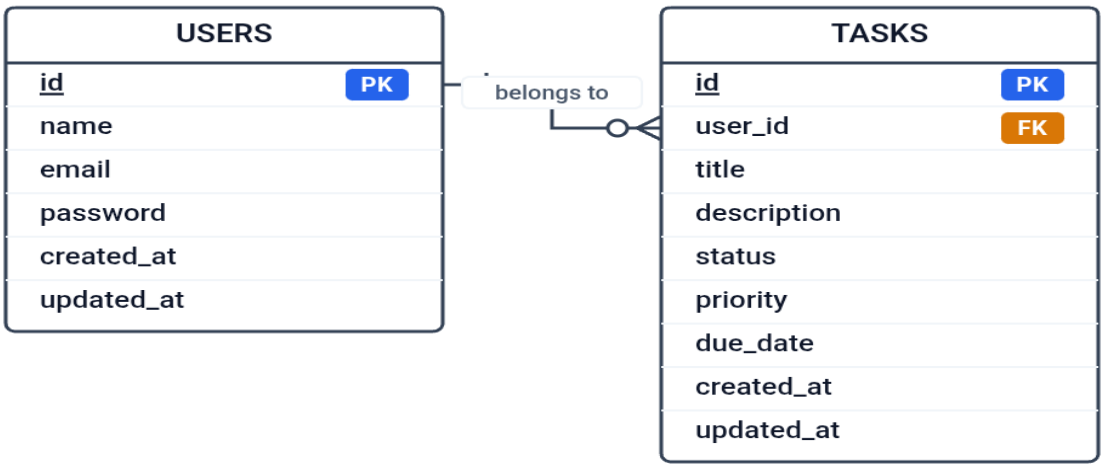

Figure 2: Entity Relationship Diagram (ER Diagram)

8. System Workflow

The Task Manager System follows a structured workflow to ensure smooth interaction between the user and the system. The workflow describes the sequence of actions performed by the user, starting from authentication until task completion.

The system is designed to provide a simple and efficient process for managing tasks while maintaining data security and ownership.

8.1 User Authentication Process

The workflow begins when the user accesses the system. The user must first register an account if they are new to the system. After registration, the user verifies their email address and then logs into the system.

This authentication process ensures secure access and prevents unauthorized users from entering the system.

8.2 Task Creation Process

After successful login, the user can create a new task by entering:

- Task title
- Task description
- Task priority
- Task status
- Due date

The system validates the input data before saving it into the database.

This process ensures data accuracy and consistency.

8.3 Task Management Process

Once tasks are created, users can manage them through different operations:

- View all tasks.
- Search for specific tasks.
- Filter tasks by status or priority.
- Edit task information.
- Delete tasks.

These operations allow users to keep their task list updated and organized.

8.4 Task Completion Process

Users can update task status when tasks are completed. This helps in tracking progress and distinguishing between completed and pending tasks.

The system also supports overdue task detection based on due dates.

8.5 Profile Management Process

The system allows users to manage their profiles, including:

- Updating profile information.
- Changing passwords.
- Deleting accounts.

This improves user flexibility and account management.

Overall, the system workflow is designed to be simple, organized, and efficient, allowing users to manage tasks with minimal complexity.

Figure 3: Use Case Diagram

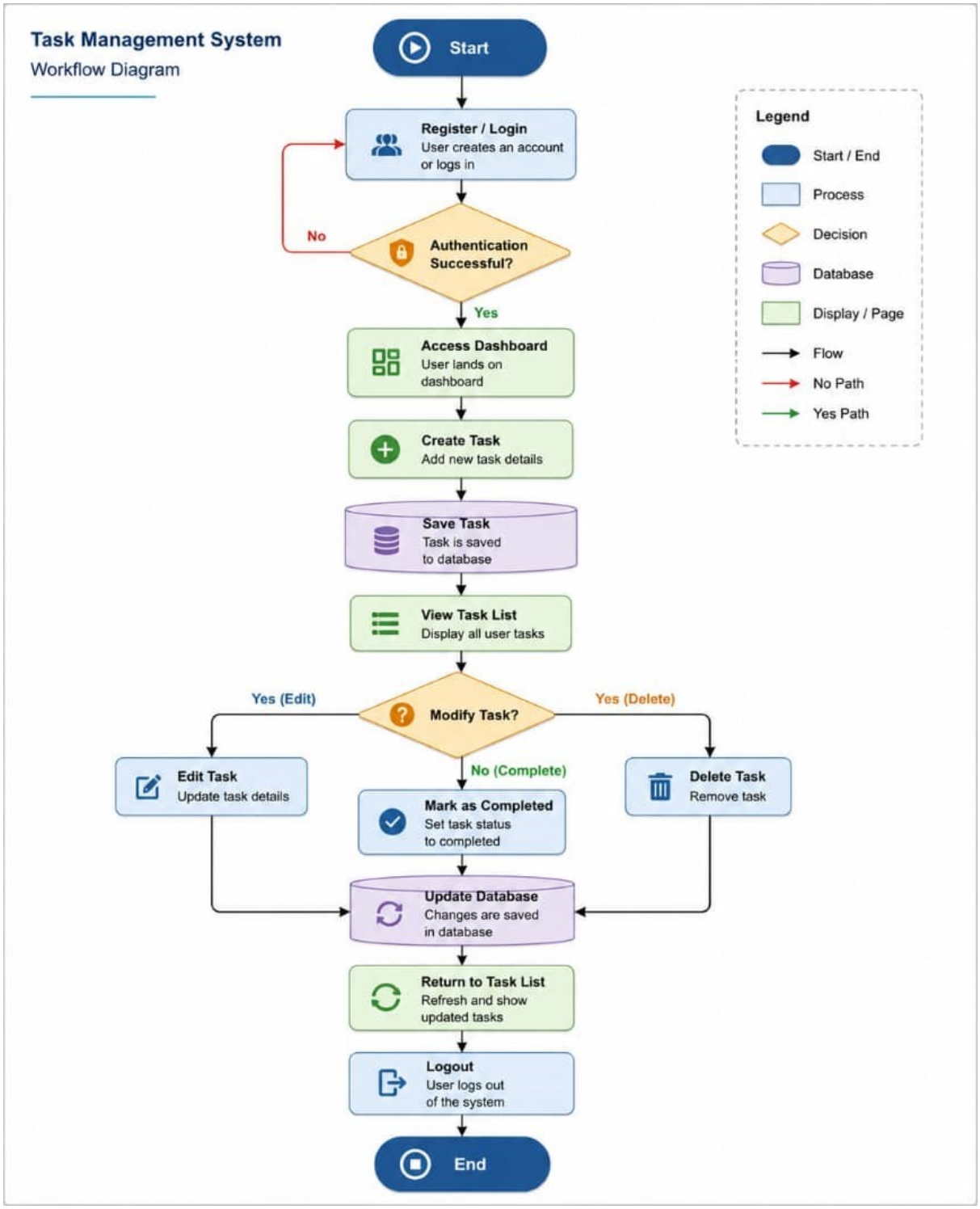

Figure 4: Workflow Diagram

9. System Interfaces

The Task Manager System provides a simple and user-friendly interface that allows users to interact with the system efficiently. The interfaces were designed to be clear, organized, and responsive to improve user experience.

The system contains several main interfaces, each serving a specific purpose in task management.

9.1 Login Interface

The Login Interface allows registered users to enter their email and password to access the system securely.

Features:

- User authentication.
- Password reset option.
- Email verification support.

This page acts as the entry point to the system.

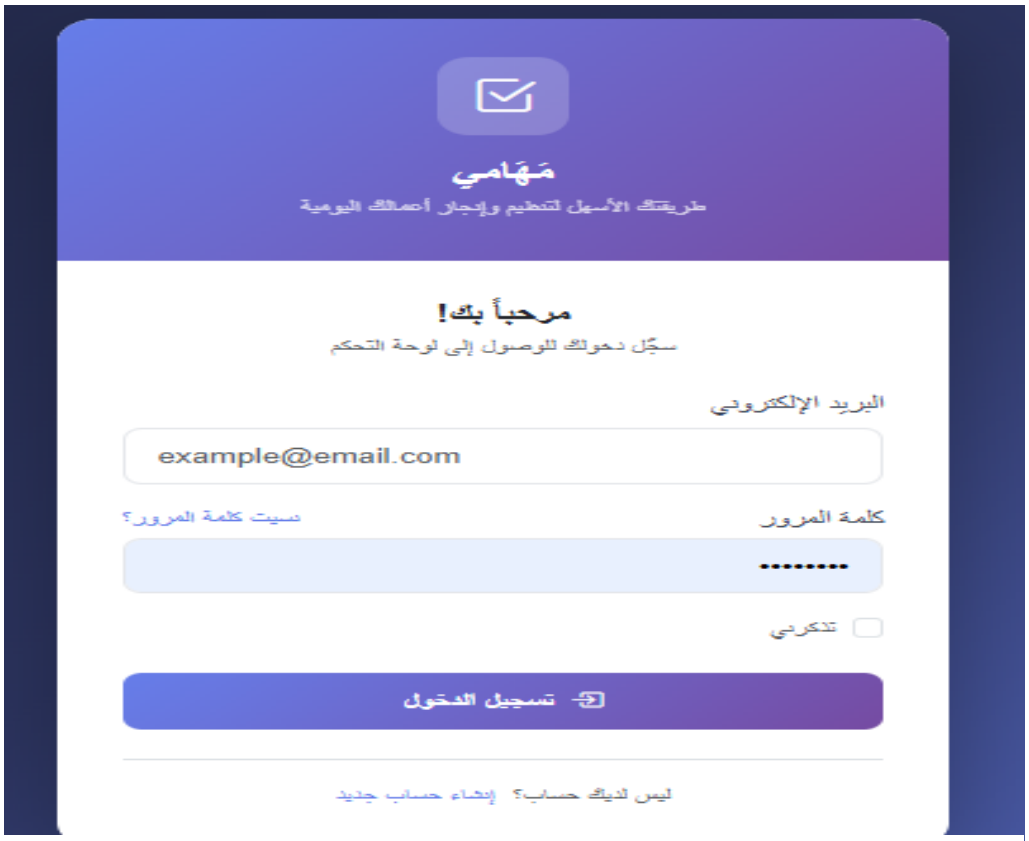

Figure 5: Login Interface

9.2 Registration Interface

The Registration Interface allows new users to create accounts by providing their personal information.

Features:

- Name input.
- Email input.
- Password creation.
- Account creation.

This interface ensures secure user registration.

Figure 6: Registration Interface

9.3 Dashboard Interface

The Dashboard provides users with an overview of their tasks and system activities.

Features:

- Quick access to task management.
- Navigation menu.
- Task statistics.

It serves as the main control panel after login.

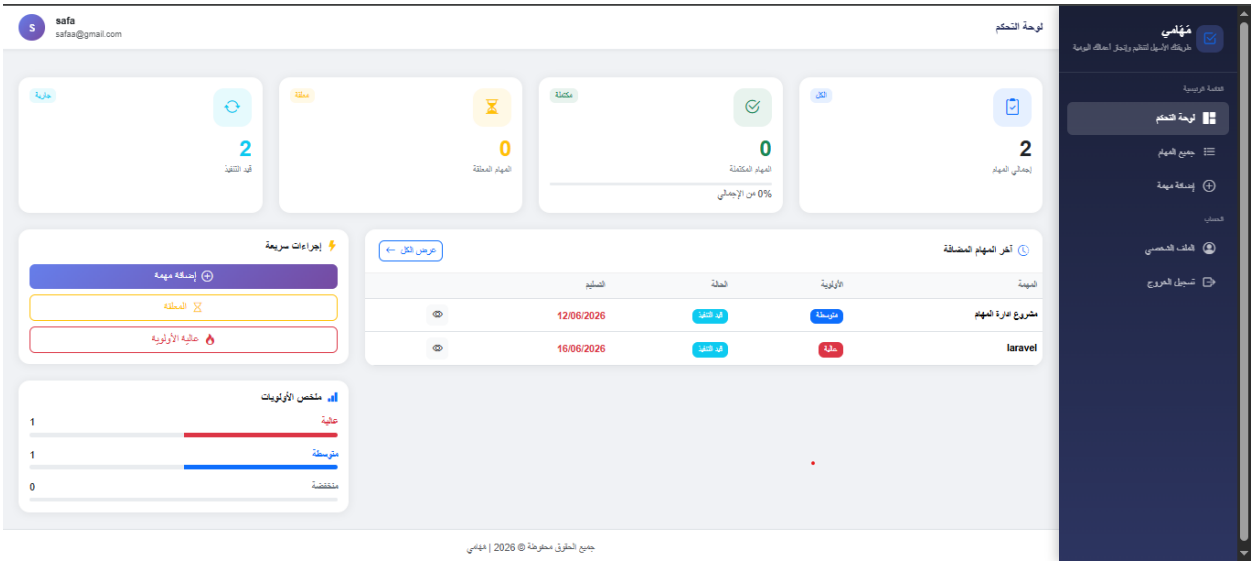

Figure 7: Dashboard Interface

9.4 Task List Interface

This interface displays all tasks created by the user.

Features:

- Task listing.
- Search functionality.
- Filtering by priority and status.
- Task details access.

It helps users organize and monitor tasks effectively.

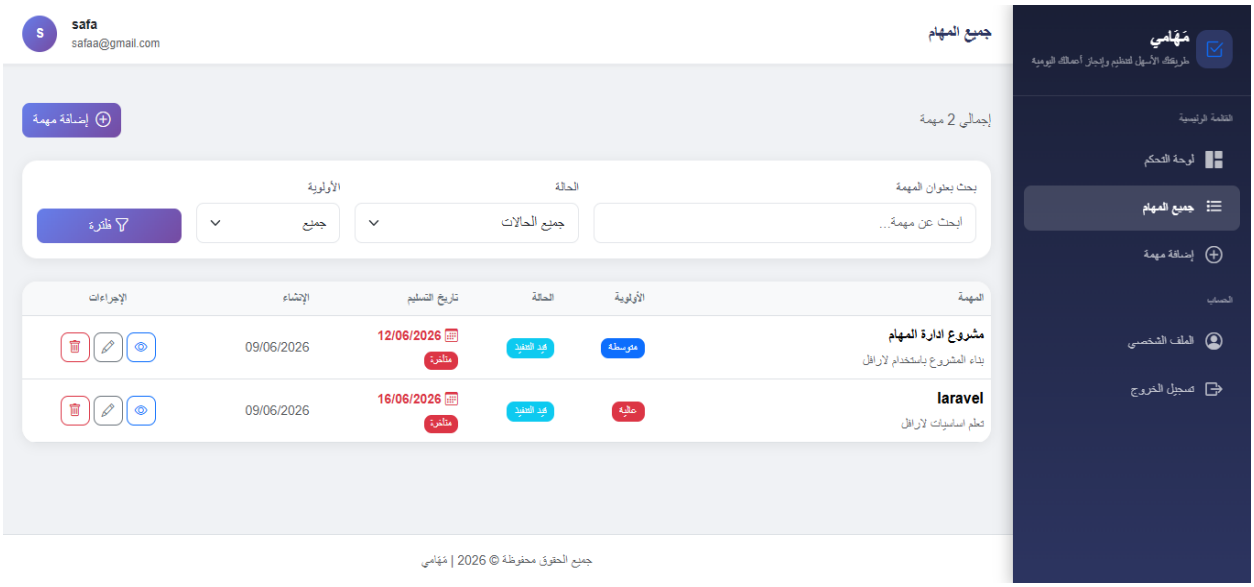

Figure 8: Task List Interface

9.5 Create Task Interface

This interface allows users to create new tasks by entering task details.

Features:

- Task title input.
- Description field.
- Priority selection.
- Status selection.
- Due date setting.

It is the main interface for task creation.

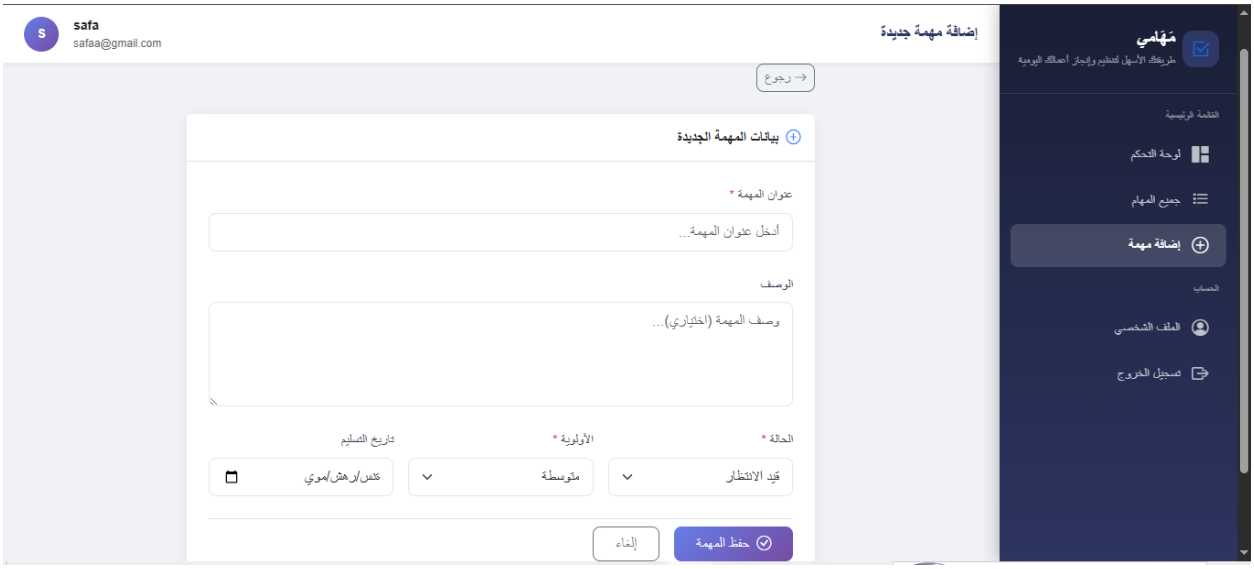

Figure 9: Create Task Interface

9.6 Edit Task Interface

The Edit Interface allows users to update existing tasks.

Features:

- Modify title.
- Update description.
- Change status.
- Update priority.
- Change due date.

This helps maintain task information accurately.

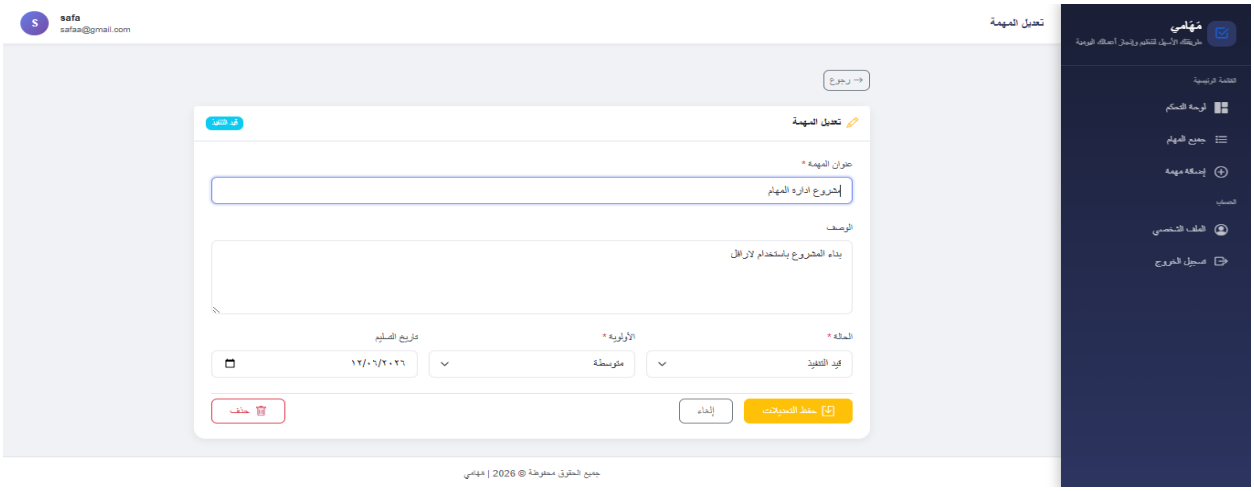

Figure 10: Edit Task Interface

9.7 Profile Management Interface

This interface allows users to manage their personal information.

Features:

- Update profile data.
- Change password.
- Delete account.

It gives users full control over their accounts.

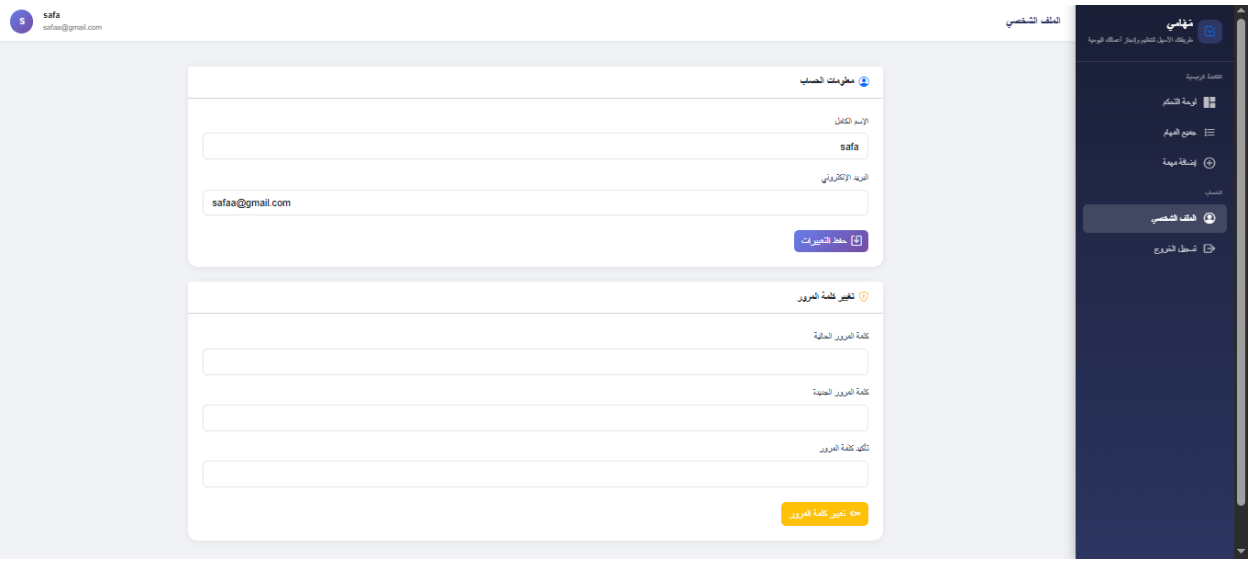

Figure 11: Profile Management Interface

The user interfaces were designed to be responsive, organized, and easy to use, ensuring smooth interaction between users and the system.

10. Future Enhancements

Although the Task Manager System successfully provides the core functionalities required for task management, there are several possible improvements that can enhance the system in the future and make it more advanced and scalable.

The following enhancements can be considered:

10.1 Email Notifications

The system can be improved by adding email reminder notifications for upcoming deadlines and overdue tasks. This feature would help users stay informed and avoid missing important tasks.

10.2 Mobile Application

A mobile version of the system can be developed for Android and iOS platforms. This would allow users to manage their tasks anytime and anywhere.

10.3 Task Sharing

Future versions can support task sharing between users, allowing collaboration on tasks and projects.

This feature would be useful for teamwork environments.

10.4 Team Management

The system can be expanded to support teams, where multiple users can work together on shared task boards.

This would transform the system from personal task management into team project management.

10.5 Calendar Integration

Integrating tasks with a calendar system would provide better visualization of deadlines and schedules.

This would improve planning and task organization.

10.6 Real-Time Notifications

Using technologies like WebSockets or Laravel Echo, the system can provide instant notifications for ask updates and deadlines.

This would improve responsiveness and user engagement.

10.7 API Development

Creating RESTful APIs would allow integration with external systems or mobile applications.

This would improve system flexibility and scalability.

10.8 Task Analytics

Future versions can include analytics dashboards showing:

- Completed tasks
- Pending tasks
- Productivity trends
- Task completion rates

This would help users evaluate their performance and improve productivity.

These enhancements would increase the system's functionality, improve user experience, and make the project more suitable for larger-scale use.

11. Conclusion

The Task Manager System successfully provides a practical and efficient solution for managing daily tasks in an organized and secure way. The system was developed using the Laravel framework and implemented following the Model-View-Controller (MVC) architecture, which improves code structure, maintainability, and scalability.

The project achieved its main objectives by allowing users to create, update, delete, search, and manage tasks based on priority, status, and deadlines. It also ensures data security through authentication, authorization, email verification, and ownership protection.

Through the implementation of this project, important software development concepts were applied, including database design, middleware, routing, validation, and secure user access control. The project demonstrates how modern web technologies can be used to solve real-life organizational problems effectively.

Although the current version of the system meets the core requirements, there is still potential for future improvements such as notifications, mobile applications, team collaboration, and advanced analytics.

Overall, the Task Manager System is a strong foundation for task management applications and represents a successful practical implementation of Laravel web development concepts.